

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 9 — TOOLS & AGENTS

Prerequisites: Phases 1–8 done; checkpointers (Phase 5) and the RAG retriever (Phase 8) fresh; `langgraph` installed.

Targets: tools (`@tool`) · the tool-calling loop · `create_agent` · agent memory · an agentic RAG tool

Estimated time for this phase: 90–130 minutes

Where this phase sits

So far the model only *answers*. Now it gets to **act**: you give it **tools** (functions it can call), and it decides — on its own, in a loop — when to use them. This is the leap from chains to **agents**, built with the modern `create_agent` API.

Ph 0–4 ✓ Setup → Fundamentals → Models → Prompts → Chains

Ph 5 ✓ Memory & Conversational Apps

Ph 6–8 ✓ Documents · Embeddings · RAG

Ph 9 **Tools & Agents** ← you are here
Defining tools, the tool-calling loop, `create_agent`, agent memory, an agentic RAG tool.

Ph 10 **LangGraph Basics** — build stateful multi-step workflows by hand.

Ph 11 **Observability & Deployment**

Ph 12 **Capstone Project**

Phase 9 · Tools & Agents

WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Explain how an agent differs from a chain; define custom tools with the `@tool` decorator; describe the tool-calling loop; build a tool-using agent with `create_agent` on both Gemini and Ollama; give it memory; and turn your Phase 8 RAG retriever into a tool the agent calls on demand.

In this phase

1. 9.1 — Chains vs agents: the leap from answering to acting
2. 9.2 — What a tool is (the `@tool` decorator)
3. 9.3 — The tool-calling loop, under the hood
4. 9.4 — Your first agent with `create_agent`
5. 9.5 — Multiple tools & watching it reason
6. 9.6 — Giving an agent memory
7. 9.7 — Agentic RAG: the retriever as a tool
8. 9.8 — The legacy agent APIs (avoid these)
9. 9.9 — Common errors & troubleshooting
10. 9.10 — Mini practice exercise
11. Terms introduced this phase

9.1 — Chains vs agents: the leap from answering to acting

Every chain you've built follows a **fixed path**: input goes in, flows through known steps, answer comes out. The route never changes. An **agent** is different: the model itself *decides* what to do at each step — which tool to call, with what inputs, and when it's done — looping until it has the answer.

TERMS: TOOL & AGENT

A **tool** is a function you give the model permission to call — a calculator, a web search, a database lookup, your RAG retriever. An **agent** is a model wrapped in a loop that lets it call tools repeatedly and reason about the results until it can answer. Chain = you decide the steps; agent = the model decides the steps.

	Chain (Phases 1–8)	Agent (this phase)
Control flow	Fixed, written by you	Decided by the model at runtime
Tools	None — just transforms data	Can call tools, see results, call more
Loops?	No (straight through)	Yes — until it's satisfied
Best for	Predictable, known pipelines	Open-ended tasks needing live actions

WHY THIS MATTERS

A chain can't answer "What's $4,891 \times 237$, and is that more than the population of Iceland?" — it has no calculator and no live data. An agent with a math tool and a search tool can: it calls the calculator, calls search, compares, and replies. The model orchestrates; the tools do the work it can't.

9.2 — What a tool is (the `@tool` decorator)

A tool is just a Python function with a clear name, type hints, and a docstring. The `@tool` decorator turns it into something the model can understand and call.

```
from langchain.tools import tool          # (also: langchain_core.tools)

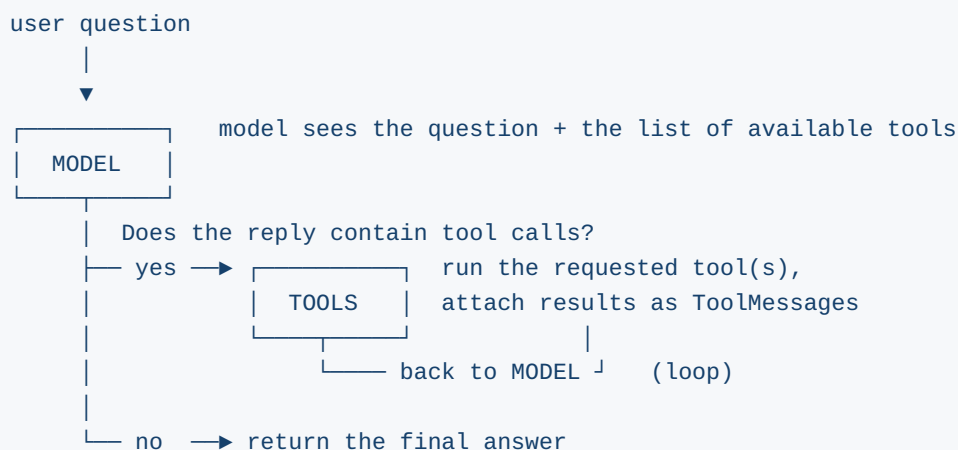
@tool
def multiply(a: int, b: int) -> int:    # type hints describe the inputs
    """Multiply two numbers together.""" # docstring tells the model what it does
    return a * b
```

THE DOCSTRING AND TYPES ARE THE INSTRUCTIONS

When the model decides whether to use a tool, it reads exactly three things the decorator extracts automatically: the **name** (`multiply`), the **description** (your docstring), and the **parameter schema** (from the type hints). So write clear docstrings and accurate type hints — they're not documentation for humans, they're the model's only guide to using the tool correctly. A vague docstring is the #1 cause of an agent misusing a tool.

9.3 — The tool-calling loop, under the hood

Agents look magical but the mechanism is simple. Here's the loop `create_agent` runs for you:



TERM: TOOL CALL & TOOLMESSAGE

When the model wants to use a tool, it doesn't run it — it emits a **tool call**: the tool's name plus arguments (e.g. `multiply(a=23, b=17)`). The agent runs the function, wraps the result in a **ToolMessage**, and

feeds it back to the model. The model then either calls more tools or writes the final answer. This back-and-forth is the "ReAct" (Reason + Act) pattern.

9.4 — Your first agent with `create_agent`

`create_agent` builds that entire loop from a model and a list of tools. One function call.

```
from dotenv import load_dotenv
from langchain.agents import create_agent          # 1 the modern entrypoint
from langchain.tools import tool
from langchain_google_genai import ChatGoogleGenerativeAI

load_dotenv()

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers together."""
    return a * b

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)

agent = create_agent(                             # 2 build the agent loop
    model,
    tools=[multiply],
    system_prompt="You are a helpful math assistant.",
)

result = agent.invoke(                           # 3 invoke with a messages list
    {"messages": [{"role": "user", "content": "What is 23 times 17?"}]}
)
print(result["messages"][-1].content)            # 4 the final answer
```

Line	Explanation
1	<code>create_agent</code> from <code>langchain.agents</code> — the current API (it builds a LangGraph agent under the hood).
2	Pass the model, a <code>tools</code> list, and an optional <code>system_prompt</code> . (The model can also be a string like <code>"google_genai:gemini-2.5-flash"</code> .) The agent now knows it <i>may</i> call <code>multiply</code> .
3	Invoke with a <code>messages</code> list — same shape as Phase 2/5. The agent runs the loop: model decides to call <code>multiply(23, 17)</code> , gets <code>391</code> , then writes the answer.
4	The result is the full message list; the last message holds the final answer.

Expected output:

```
23 times 17 is 391.
```

The **local** version swaps only the model — and note local models must support tool calling (`llama3.2` does):

```
from langchain_ollama import ChatOllama
model = ChatOllama(model="llama3.2", temperature=0)
agent = create_agent(model, tools=[multiply], system_prompt="You are a helpful math assistant.")
```

9.5 — Multiple tools & watching it reason

Give the agent several tools and it'll pick the right one(s). To see its reasoning, print every message in the result — you'll watch the loop happen.

```
@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

@tool
def word_length(word: str) -> int:
    """Return the number of letters in a word."""
    return len(word)

agent = create_agent(model, tools=[add, multiply, word_length])

result = agent.invoke({"messages": [
    {"role": "user", "content": "How many letters in 'banana', and what's that number times 10?"}
]})

for msg in result["messages"]:
    msg.pretty_print()           # nicely shows each turn of the loop
```

Expected output (abridged) — note it chains two tools on its own:

```
===== Human =====
How many letters in 'banana', and what's that number times 10?
===== Ai =====
Tool Calls: word_length(word='banana')
===== Tool =====
6
===== Ai =====
Tool Calls: multiply(a=6, b=10)
===== Tool =====
60
===== Ai =====
'banana' has 6 letters, and 6 times 10 is 60.
```

THIS IS THE WHOLE POINT OF AGENTS

You never told it "first count letters, then multiply." It worked out the plan, called `word_length`, used the result to call `multiply`, and composed the answer. With chains you'd have to wire that sequence yourself; the agent figures it out from the tools and the question.

9.6 — Giving an agent memory

Agents use the **same memory mechanism as Phase 5**: pass a `checkpointer` and invoke with a `thread_id`. The agent then remembers across turns automatically.

```
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(model, tools=[multiply], checkpointer=InMemorySaver()) # add
memory

config = {"configurable": {"thread_id": "user-1"}}

agent.invoke({"messages": [{"role": "user", "content": "Multiply 6 and 7."}]}, config)
r = agent.invoke({"messages": [{"role": "user", "content": "Now add 8 to that
result."}]}, config)
print(r["messages"][-1].content) # knows "that result" was 42 -> 50
```

EVERYTHING COMPOSES

Memory (Phase 5) drops straight into agents because `create_agent` builds on the same LangGraph foundation. One `checkpointer=` argument and a `thread_id` — identical to the chatbot you already built. This consistency is the reward for learning the pieces in order.

9.7 — Agentic RAG: the retriever as a tool

In Phase 8 the RAG chain *always* retrieved. But what if some questions don't need the documents (e.g. "hello") and others need several searches? Make retrieval a **tool** and let the agent decide when — and how often — to search. This is "agentic RAG."

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_community.vectorstores import FAISS
from langchain_google_genai import ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings

load_dotenv()

embeddings = GoogleGenerativeAIEmbeddings(model="gemini-embedding-001")
store = FAISS.load_local("faiss_index", embeddings, allow_dangerous_deserialization=True)
retriever = store.as_retriever(search_kwargs={"k": 3})

@tool
def search_handbook(query: str) -> str: # the retriever, wrapped as a
    tool
    """Search the company handbook for relevant information.
```

```

    Use this for any question about company policies."""
    docs = retriever.invoke(query)
    return "\n\n".join(d.page_content for d in docs)

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)
agent = create_agent(
    model,
    tools=[search_handbook],
    system_prompt="Answer questions about the handbook. Use search_handbook when
needed.",
)

r = agent.invoke({"messages": [{"role": "user", "content": "What is the refund
policy?"}]})
print(r["messages"][-1].content)

```

CHAIN-RAG VS AGENTIC-RAG

Phase 8's **chain** RAG is simpler and predictable — best when every query needs the docs. **Agentic** RAG is more flexible — the agent can skip retrieval for chit-chat, search multiple times, or combine the handbook tool with other tools (a calculator, web search). Start with chain RAG; reach for agentic RAG when questions vary widely or need multiple steps.

9.8 — The legacy agent APIs (avoid these)

⚠ LEGACY — DO NOT USE IN NEW CODE

Agents changed a lot, so search results are full of dead patterns. If you see these, replace them with `create_agent` :

Deprecated / legacy	Use instead
<code>AgentExecutor</code>	<code>create_agent</code>
<code>initialize_agent(...)</code>	<code>create_agent</code>
<code>create_react_agent</code> (old <code>langchain.agents</code> one), <code>create_tool_calling_agent</code>	<code>create_agent</code>
<code>ZeroShotAgent</code> , <code>ChatAgent</code> , hub ReAct prompts + <code>agent_scratchpad</code>	<code>create_agent</code> (handles the loop internally)

The tell: legacy code builds an `agent` object and then wraps it in an `AgentExecutor` , often with a hand-written prompt containing `{agent_scratchpad}` . The modern way needs none of that.

9.9 — Common errors & troubleshooting

Symptom	Likely cause & fix
---------	--------------------

Agent never calls the tool / answers from memory instead	Weak tool docstring, or the question doesn't obviously need it. Improve the docstring; nudge via <code>system_prompt</code> .
Ollama: agent ignores tools or errors on tool calls	The model must support tool calling. Use <code>llama3.2</code> (does) — some small models don't. Check the model's page.
Tool called with wrong/garbled arguments	Type hints unclear or missing. Add precise hints (<code>a: int</code>) and describe each in the docstring.
<code>ImportError: create_agent</code>	Old/incomplete install. <code>pip install -U langchain langgraph</code> (1.x). It lives in <code>langchain.agents</code> .
Agent loops forever / many tool calls	Ambiguous task or a tool that never satisfies it. Tighten the prompt; agents have a step limit but a bad tool can still thrash.
Memory not working	Missing <code>checkpointers=</code> or inconsistent <code>thread_id</code> — same fixes as Phase 5.
You followed a tutorial with <code>AgentExecutor</code> and it errors	That's legacy (§9.8). Rebuild with <code>create_agent</code> .

9.10 — Mini practice exercise

YOUR TURN (DO THIS BEFORE PHASE 10)

Task: Build a multi-tool assistant. Define at least three tools: a calculator tool (or two — `add` , `multiply`), a `current_time` tool (use Python's `datetime`), and your `search_handbook` RAG tool from §9.7. Create one agent with all of them, add an `InMemorySaver` checkpointers, and run a short conversation that forces it to use different tools across turns — e.g. ask the time, then a handbook question, then a follow-up that references the earlier answer ("and multiply that refund window by 2"). Print the full message list once so you can see which tools fired. Try it on Gemini and Ollama.

Why: this is a real, capable assistant — exactly the shape of the Phase 12 capstone. It proves you can give a model live capabilities, let it choose among them, remember context, and ground answers in your data, all at once.

Hints (not the full answer):

- Write a crisp one-line docstring per tool — that's what the model reads to choose.
- `current_time` takes no arguments; just `return datetime.now().isoformat()` inside the function.
- Pass the same `thread_id` every turn so "that refund window" resolves from memory.
- Use `msg.pretty_print()` in a loop over `result["messages"]` to watch the reasoning.
- If Ollama is flaky with tools, confirm you're on `llama3.2` and keep tool inputs simple.

Terms introduced this phase

Term	One-line meaning
Tool	A function you allow the model to call (calculator, search, retriever, ...).
@tool	Decorator turning a typed, docstringed function into a model-callable tool.
Agent	A model wrapped in a loop that can call tools and reason until it answers.
Tool call	The model's request to run a tool with specific arguments (it doesn't run it itself).
ToolMessage	The tool's result, fed back to the model to continue reasoning.
ReAct loop	Reason → Act → observe → repeat; the cycle agents run.
create_agent	The modern API that builds the whole agent loop from a model + tools.
system_prompt (agent)	Standing instructions shaping how the agent approaches tasks.
Agentic RAG	Retrieval exposed as a tool, so the agent decides when/whether to search.
Legacy agent APIs	<code>AgentExecutor</code> , <code>initialize_agent</code> , etc. — superseded by <code>create_agent</code> .

PHASE 9 COMPLETE

You can define tools, you understand the reason-act loop, and you can build memory-enabled, tool-using agents with `create_agent` on both backends — including turning your RAG retriever into a tool. You also know to avoid the legacy `AgentExecutor` family. **Next — Phase 10: LangGraph Basics**, where we open up the engine under `create_agent` and build stateful multi-step workflows *by hand* — nodes, edges, conditional branching, and loops — for the times you need more control than a prebuilt agent gives.